SHADERMESH: A DECENTRALIZED MARKETPLACE FOR RENTING GPU AND LLM COMPUTE ON ETHEREUM

Project Report: Proposal and Design

Submitted by

Name	Roll number
Rakesh Kumar Rabha	24BCC7002
Nilam Sikdar	24BCC7025
Animesh Mishra	24BCC7016

Under the guidance of [GUIDE NAME], [GUIDE DESIGNATION]



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

VIT-AP UNIVERSITY, AMARAVATI, ANDHRA PRADESH

OCTOBER 2026

CONTENTS

1. PROJECT TITLE 3

2. TEAM MEMBERS AND ROLL NUMBERS 3

3. PROBLEM STATEMENT 3

4. OBJECTIVES 4

5. MOTIVATION AND NEED FOR THE PROJECT 5

6. LITERATURE SURVEY 5

7. EXISTING SYSTEM 7

8. PROPOSED SYSTEM 7

9. BLOCKCHAIN CONCEPTS USED 9

10. SYSTEM ARCHITECTURE AND BLOCK DIAGRAM 10

11. MODULES 11

12. TECHNOLOGY STACK 13

13. METHODOLOGY AND WORKFLOW 14

14. DATASET AND INPUT DETAILS 17

15. EXPECTED OUTCOMES 17

16. REFERENCES 19

1. PROJECT TITLE

ShaderMesh: A Decentralized Marketplace for Renting GPU and LLM Compute on Ethereum

ShaderMesh lets people with idle GPUs rent them to people who need compute, without a company in the middle holding the money. Ethereum smart contracts hold the escrow and the providers' collateral, pay providers one interval at a time, and penalize providers that stop serving. The workloads themselves, such as GPU containers and LLM inference, run off chain on the provider's machine.

2. TEAM MEMBERS AND ROLL NUMBERS

Table 2.1 lists the team members.

TABLE 2.1: Team members

S. No.	Name	Roll number
1	Rakesh Kumar Rabha	24BCC7002
2	Nilam Sikdar	24BCC7025
3	Animesh Mishra	24BCC7016

Guide: [GUIDE NAME], [GUIDE DESIGNATION], School of Computer Science and Engineering, VIT-AP University.

3. PROBLEM STATEMENT

3.1 What problem is being addressed

Demand for GPU compute has grown quickly with large language models, and cloud GPUs are expensive and often unavailable. At the same time many capable GPUs, in gaming PCs, laptops, and small labs, sit idle most of the day. Renting that capacity between strangers is hard for three reasons:

- **Payment risk.** The renter does not want to pay before the hours are served, and the owner does not want to serve before being paid.
- **No evidence of service.** Nothing records whether the GPU was actually available for each hour that was billed.

- **No enforcement.** If a provider goes offline or misrepresents its hardware, the renter has no recourse except a central platform's support desk.

Existing GPU marketplaces solve this by acting as the trusted operator: they hold funds, keep reputation scores, and decide disputes. Fig. 3.1 contrasts that model with the approach taken in this project.

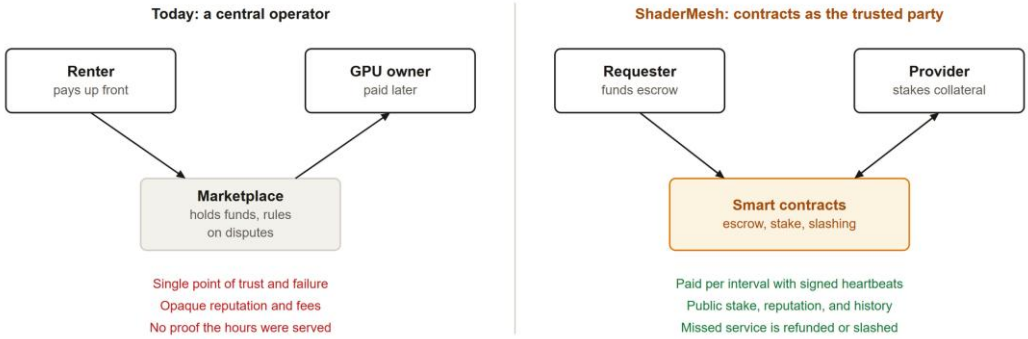


Fig. 3.1: A central operator compared with contracts as the trusted party

3.2 Why blockchain and cryptocurrency are relevant

- **Escrow without a custodian.** A smart contract can hold the renter's deposit and release it per interval under fixed rules that neither party can change.
- **Collateral that can be slashed.** Providers lock tokens as stake. A contract can take part of that stake when a provider fails, which gives the provider a reason to stay honest.
- **A shared, public record.** Leases, heartbeats, payouts, challenges, and slashes are events on a public chain, so reputation is computed from history that anyone can check.
- **Programmable payments.** A token lets the contract pay small amounts every interval and refund the unused remainder automatically.

The blockchain is used only where shared economic state and deterministic enforcement are needed. Compute stays off chain, because executing GPU workloads on chain is neither possible nor useful.

4. OBJECTIVES

1. Design and implement smart contracts that hold rental escrow and provider stake, stream payment to providers per verified interval, refund unused time, and slash providers that breach the service terms.

2. Build a trust model for compute that cannot be re-executed, based on signed per-interval heartbeats, hardware attestation, and seeded benchmark challenges that are judged and enforced on chain.
3. Deliver real GPU and LLM compute through the marketplace: GPU containers run by a provider agent with Docker, and an OpenAI-compatible inference endpoint paid through prepaid credit.
4. Provide a web application in which requesters rent capacity and providers stake and list hardware, with every value-moving action signed by the user's own wallet.
5. Deploy the system to the Ethereum Sepolia test network, run it with a real GPU provider, and evaluate it with automated tests, gas measurements, and an economic simulation.

5. MOTIVATION AND NEED FOR THE PROJECT

- **Cost and access.** Students, researchers, and small teams need GPUs for model inference and fine-tuning but cannot always pay cloud prices. Idle consumer GPUs can serve many of these workloads.
- **Trust between strangers.** A peer-to-peer market only works if both sides are protected. Escrow and stake replace the trust normally placed in a platform company.
- **A gap in existing systems.** Most decentralized compute projects either verify results by re-execution, which does not work for interactive rentals, or rely on a central operator for disputes. A rental market that is secured by availability evidence and slashing is less explored.
- **Learning value.** The project covers the full stack of a blockchain application: contract design and security, off-chain services, wallet-based user interfaces, testnet deployment, and economic analysis.

6. LITERATURE SURVEY

The survey covers decentralized compute markets, verification protocols, and centralized GPU platforms. Table 6.1 summarizes each work, its approach, and its limitation for the problem addressed here.

TABLE 6.1: Related systems and papers

Work	Approach	Limitation or gap
Golem [1]	Peer-to-peer market of requestors and providers, paid in GLM tokens	Result checking is left to each application; no general availability guarantee for rentals
iExec [2]	Proof-of-Contribution: replicated execution by worker pools, optional TEEs	Replication multiplies cost; designed for tasks with comparable outputs
Akash Network [3]	Reverse auction for container leases on a Cosmos chain, escrow per block	Payment continues on the provider's word; no on-chain evidence that each block was served
Render Network [4]	GPU rendering market with node tiers and a job manager	Job routing and quality checks depend on a central manager
Livepeer [5]	Staked transcoders with probabilistic micropayments and segment checks	Specific to video transcoding; checks rely on deterministic output
Truebit [6]	Interactive verification game that resolves disputes over computation	Needs deterministic, re-executable computation; complex dispute rounds
Filecoin [7]	Proofs of replication and spacetime for storage, with slashing	Proofs are specific to storage, not to GPU availability
Bittensor [8]	Validators score miners on model quality and split token rewards	Scores are subjective and set by validators; no escrowed rental
Arbitrum [9]	Optimistic execution with interactive fraud proofs	Targets deterministic contract execution, not external hardware
Vast.ai [10]	Centralized GPU rental with platform reliability scores	The platform holds funds and decides disputes

6.1 Existing approaches

Three approaches appear in the literature. Re-execution and replication (iExec, Truebit, Livepeer) check that a result is correct by computing it again. Proofs specific to a resource (Filecoin) bind rewards to a cryptographic proof of service. Operator trust (Render, Vast.ai, and in practice many token-based markets) keeps a company in charge of routing, quality, and disputes.

6.2 Limitations and gaps

- A rented GPU session is interactive and non-deterministic, so it cannot be re-executed and compared.
- Lease payments in existing decentralized markets usually continue without per-interval evidence that the provider was online.
- Hardware claims are rarely re-checked after registration, so a provider can advertise a GPU it does not serve.

- Where slashing exists, it is often tied to a single resource type, and failure costs are not separated from the stake that backs active leases.

ShaderMesh addresses these gaps with per-interval heartbeats that gate each payout, recurring benchmark challenges with on-chain slashing of free stake, and a separate verification game kept only for tasks that can be recomputed.

7. EXISTING SYSTEM

7.1 Current method

Today a person who needs a GPU rents it from a cloud provider or from a centralized marketplace such as Vast.ai. The renter creates an account, pays the platform by card or prepaid balance, and receives access to a machine. The platform pays the host later, after taking a fee. Hardware and reliability scores are kept by the platform, and disputes go to its support team.

7.2 Problems and limitations

Table 7.1 lists the main limitations of this model.

TABLE 7.1: Limitations of the existing system

Limitation	Effect
Central custody of funds	The platform can freeze, delay, or lose payments; both sides must trust it
Opaque reputation	Hosts and renters cannot verify how scores are computed or what history they reflect
No proof of service	Billing is based on the platform's own records, which the renter cannot check
Weak penalties	A host that goes offline usually loses a rating, not money, so the renter bears the loss
Single point of failure	An outage or policy change at the platform stops the whole market
Fees and access barriers	Platform fees and regional payment restrictions raise the cost of compute

8. PROPOSED SYSTEM

8.1 Proposed solution

ShaderMesh moves the trusted parts of the marketplace into smart contracts and leaves only untrusted coordination off chain. The system offers two market modes over one shared staking, reputation, and slashing layer:

- **Compute rental (primary).** A requester leases a GPU for a number of fixed intervals and funds escrow for all of them. The provider accepts and bonds part of its stake. For each interval the provider's agent sends a signed heartbeat, and only intervals with a heartbeat are paid. A missed interval ends the lease, refunds the remaining escrow, and slashes the bond.
- **Managed inference.** A provider publishes a model offer. A requester opens prepaid credit and sends OpenAI-compatible requests, each signed by the requester's wallet. Usage is settled on chain against the credit.
- **Verifiable tasks (retained).** Deterministic jobs, such as rendering frames, are checked by an optimistic verification game with spot checks and slashing, because their output can be recomputed.

To keep hardware claims honest after registration, an attester issues seeded **benchmark challenges**. The provider must return the result of a 500,000-step SHA-256 hash chain and a measured throughput score within the answer window. A wrong hash, a low score, or a missed deadline slashes 10 SMESH of the provider's free stake to a treasury, revokes its attestation, and lowers its reputation. Fig. 8.1 shows how tokens move between the parties.

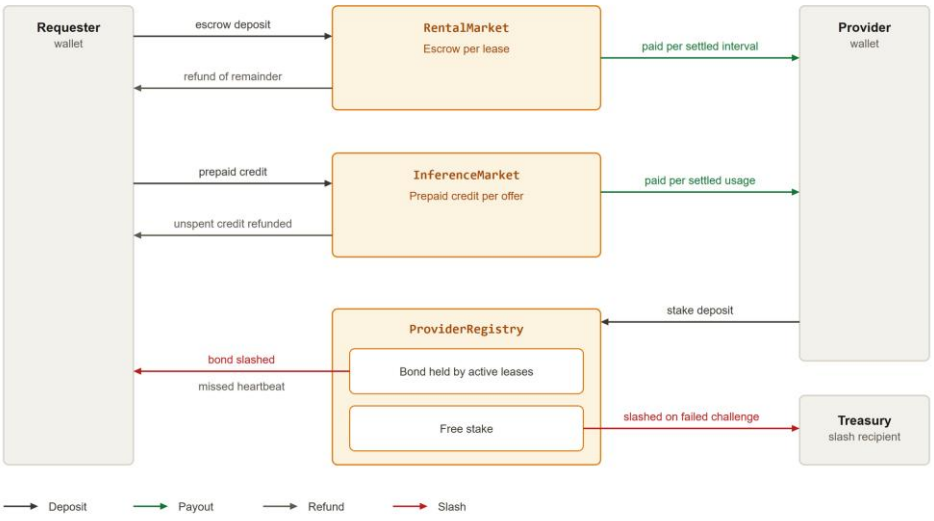


Fig. 8.1: Token flows: deposits, per-interval payouts, refunds, and slashes

8.2 Advantages

- Requester funds are held by a contract, not a company, and are released one interval at a time.

- A provider is paid only for intervals it proves with a signed heartbeat, and unused escrow is refunded automatically.
- Providers have money at risk: lease bonds for service failures and free stake for failed benchmark challenges.
- Reputation, payouts, challenges, and slashes are public events that anyone can audit in the explorer.
- Off-chain services hold no requester funds, so a compromised orchestrator cannot steal escrow.
- The same layer supports GPU containers, LLM inference, and verifiable batch jobs.

9. BLOCKCHAIN CONCEPTS USED

Table 9.1 summarizes the blockchain concepts used in the project.

TABLE 9.1: Blockchain concepts used

Concept	Choice in ShaderMesh
Blockchain type	**Public, permissionless.** Ethereum, deployed on the Sepolia test network (chain ID 11155111) and on a local Hardhat chain for development. Anyone can become a provider or requester.
Consensus mechanism	**Proof of Stake** [11]. Sepolia, like Ethereum mainnet, finalizes blocks through validators that stake ETH. ShaderMesh also uses staking at the application level: providers stake SMESH as collateral.
Smart contracts	Six Solidity 0.8.24 contracts built on OpenZeppelin [12]: SmeshToken, ProviderRegistry, RentalMarket, InferenceMarket, JobMarket, and Verification. They hold all value and enforce every state transition.
Token	**SMESH**, an ERC-20 utility token [13] used for escrow, stake, bonds, credit, and slashing. A faucet issues test tokens on Sepolia. Gas is paid in Sepolia ETH.
Cryptographic signatures	Wallet signatures authorize every transaction. EIP-191 signed messages [14] authorize inference requests off chain, and heartbeats are transactions signed by the provider key.
Hashing	SHA-256 [15] hash chains for benchmark challenges, keccak256 for challenge seeds and result commitments, and IPFS content identifiers [16] for workload manifests.
Events and indexing	Contracts emit events for every state change. The orchestrator indexes them into PostgreSQL to serve the web application.

The contracts are source-verified on Etherscan at the addresses in Table 9.2.

TABLE 9.2: Sepolia contract addresses

Contract	Address
SmeshToken	`0x4d08515811A8b14A69aE3049278EDB04e83645F1`
ProviderRegistry	`0xd723DFd9E63632e9533ea1946b57AdB6d00cfC3f`
JobMarket	`0x9F3B0526A9c665F1Df423145eaACCe5b2479Ee57`
Verification	`0x994927B22613bCB8c5A25fA6386E386F250D365F`
RentalMarket	`0x0CcaB81947Ee0Dc270908FE0cd99f280596f3Ac3`
InferenceMarket	`0x108fF3eaeB81592A04212F52bEe50F46a0abf9D1`

10. SYSTEM ARCHITECTURE AND BLOCK DIAGRAM

Fig. 10.1 shows the containers and the boundary between components that hold value and components that only coordinate. The browser wallet signs every value-moving action. The provider agent runs workloads and sends heartbeats. The attester judges benchmark challenges. The orchestrator indexes events and routes inference, but holds no funds.

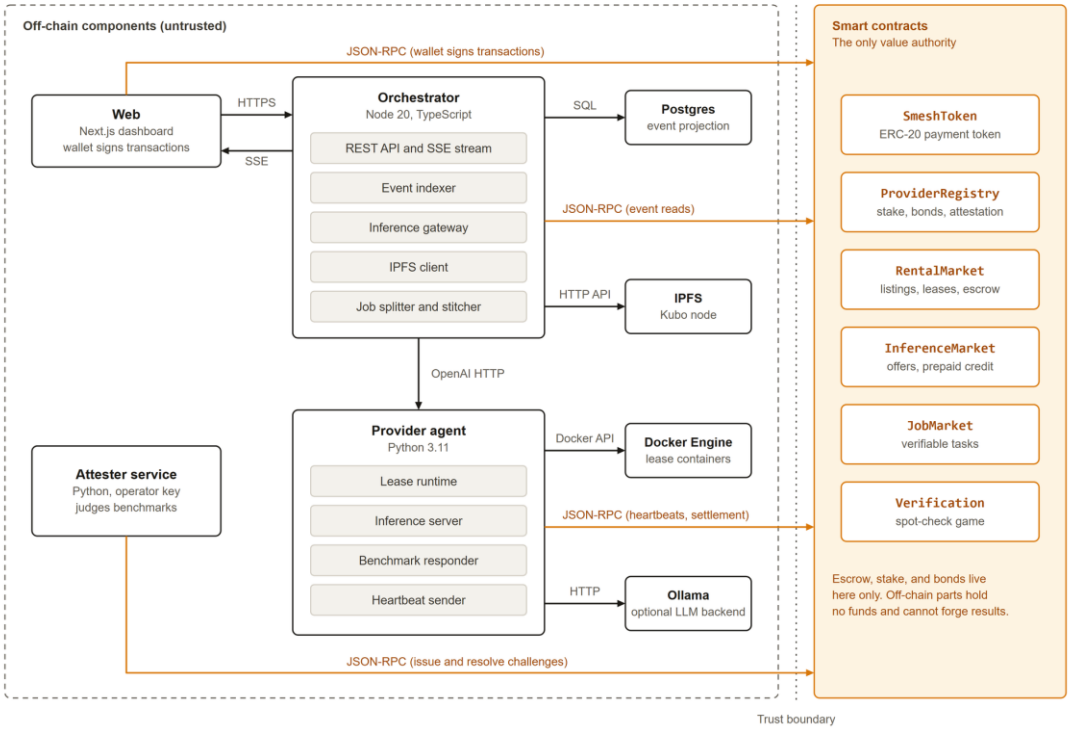


Fig. 10.1: System architecture: off-chain services and the contracts that hold all value

Fig. 10.2 states what each component can and cannot do. The rule behind the design is that a component outside the contracts may fail or lie without being able to move funds it does not own.

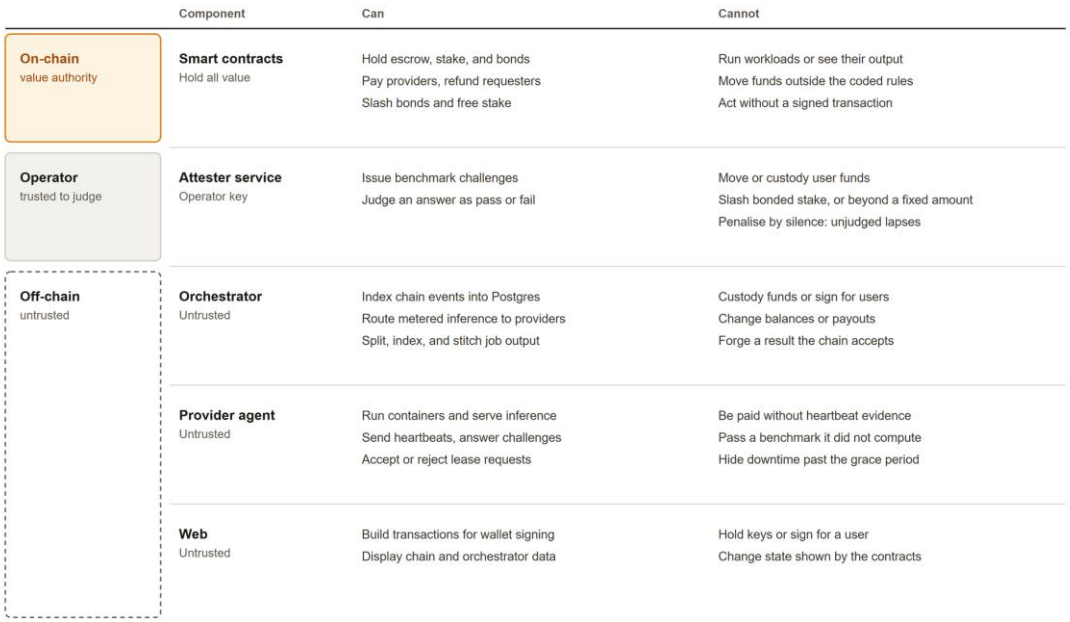


Fig. 10.2: Trust boundary: what each component can and cannot do

11. MODULES

The system is split into ten modules, shown in Fig. 11.1: five on chain and five off chain.

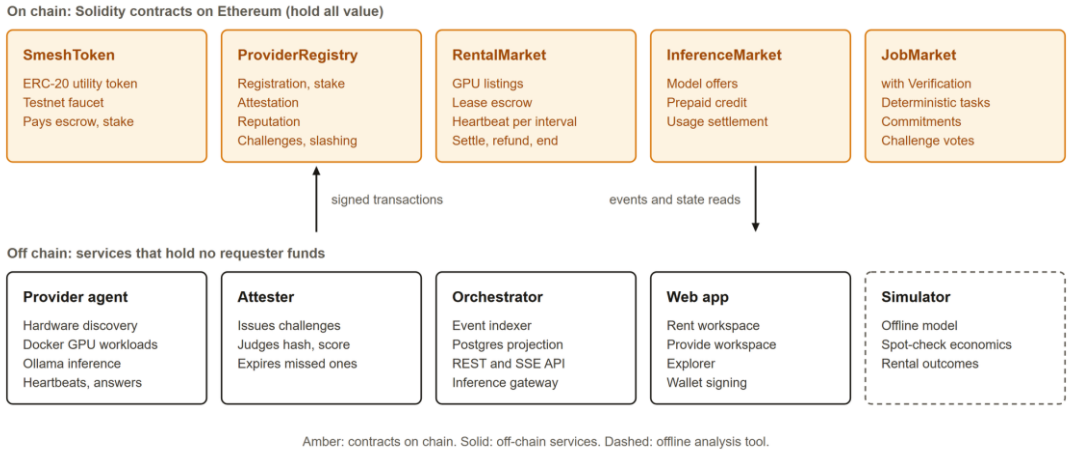


Fig. 11.1: Module map

11.1 On-chain modules

- **SmeshToken.** ERC-20 token with a rate-limited faucet for test networks.
- **ProviderRegistry.** Registers providers with their hardware capabilities, holds stake, records attestations, keeps an integer reputation updated by an exponential moving average, and runs the benchmark challenge state machine

(issue, respond, resolve, expire). It slashes only free stake, never stake that backs an active lease.

- **RentalMarket.** Holds GPU listings. A lease is requested with escrow and a workload manifest, accepted by the provider with a bond, kept alive by one heartbeat per interval, settled interval by interval, and ended by the requester, by expiry, or by a missed heartbeat.
- **InferenceMarket.** Holds model offers and prepaid credit. Usage is settled as cumulative token counts, and the requester can close the credit to recover the unused balance.
- **JobMarket and Verification.** Deterministic tasks with bonded claims, result commitments, a challenge window, and a vote that slashes the losing side.

11.2 Off-chain modules

- **Provider agent (Python).** Discovers the GPU, accepts allowed lease manifests, runs workloads in constrained Docker containers with GPU passthrough, serves inference through Ollama, sends heartbeats, and answers benchmark challenges.
- **Attester (Python).** Issues a challenge to each attested provider at a fixed interval, checks the hash and score, and records the result on chain. Anyone can expire a challenge that was not answered in time.
- **Orchestrator (TypeScript).** Indexes contract events into PostgreSQL, serves REST and server-sent events to the web application, and routes signed inference requests to the provider that holds the offer.
- **Web application (Next.js).** A landing page and an app with three workspaces: Rent (Marketplace, My leases, Inference), Provide (Node, Capacity, Stake), and an Explorer for leases, challenges, and events.
- **Simulator (Python).** A Monte Carlo model of the economics, used to choose spot-check rates and to check that rental escrow is always conserved.

Fig. 11.2 and Fig. 11.3 show the web application running against the live Sepolia deployment.

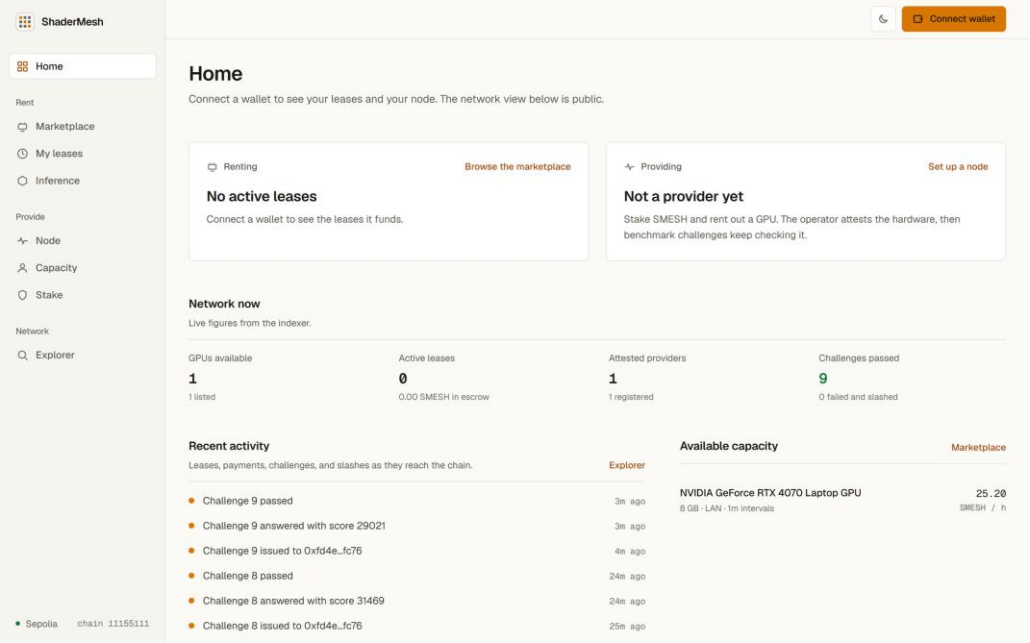


Fig. 11.2: Web application: Home page with the Renting and Providing panels

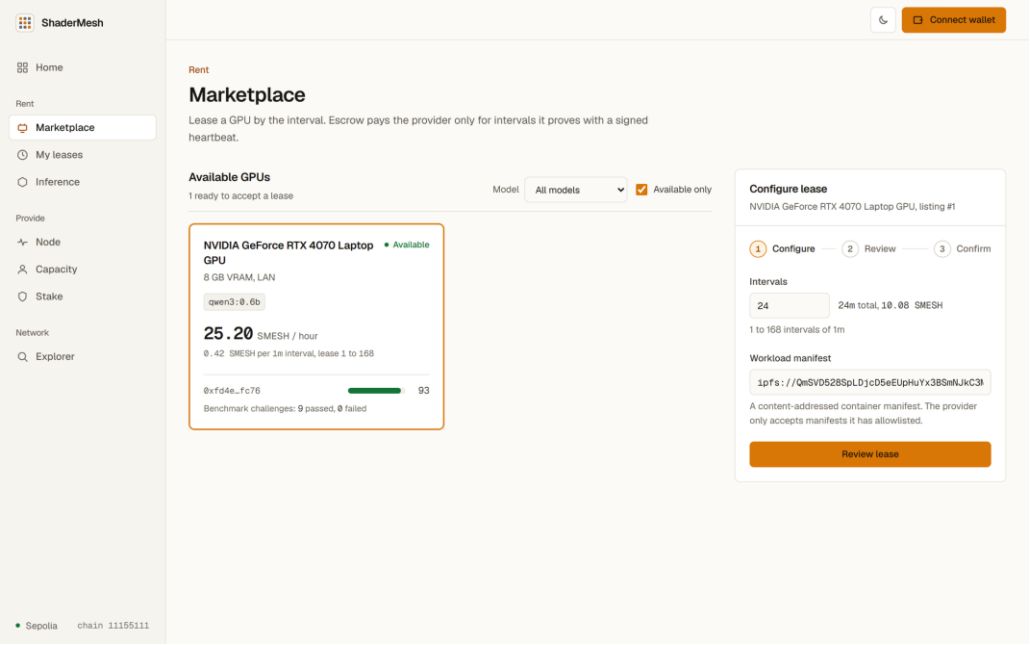


Fig. 11.3: Web application: Marketplace with the lease composer

12. TECHNOLOGY STACK

Table 12.1 groups the technologies by layer, and Table 12.1 lists them under the headings of the template.

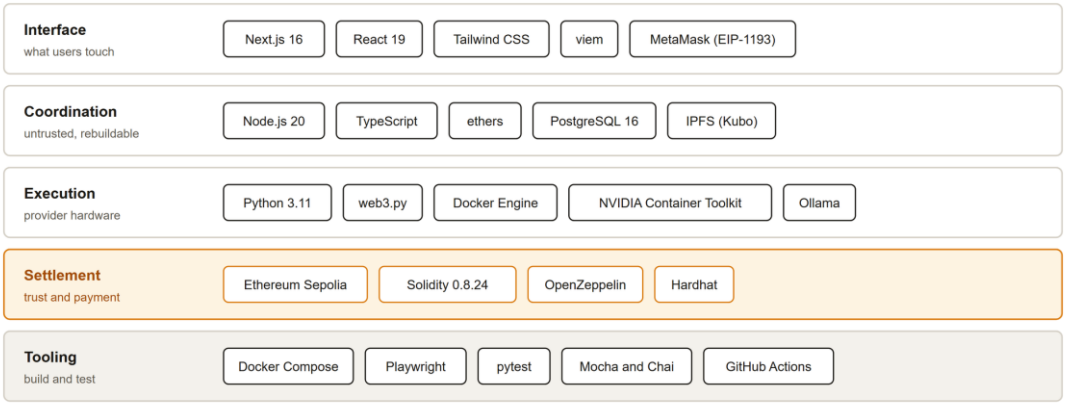


Fig. 12.1: Technology stack by layer

TABLE 12.1: Technology stack

Category	Technology
Blockchain platform	Ethereum (Sepolia test network), local Hardhat network for development
Programming languages	Solidity 0.8.24 (contracts), Python 3.11 (agent, attester, simulator), TypeScript (orchestrator, web)
Database and storage	PostgreSQL 16 (event projection), IPFS (workload manifests and job data)
Frameworks and libraries	OpenZeppelin, Hardhat [17], Next.js 16 [18], React 19, Tailwind CSS, viem [19], ethers, web3.py [20]
Compute runtime	Docker Engine with NVIDIA GPU passthrough [21], Ollama [22] for LLM inference
Wallet	MetaMask through the EIP-1193 provider interface [23]
Testing and delivery	Mocha and Chai, pytest, Playwright [24], Docker Compose, GitHub Actions

13. METHODOLOGY AND WORKFLOW

13.1 Development methodology

The project was built in seven phases (Fig. 13.1). Each phase delivered working code with automated tests before the next one started, and findings from testing were fed back into the requirements. Contracts came first because every other component depends on their interfaces and events.

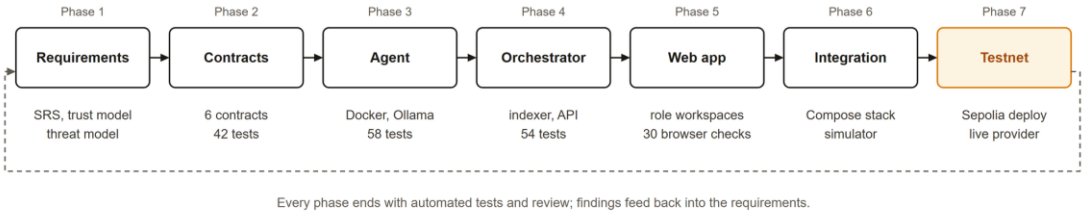


Fig. 13.1: Development phases

13.2 Rental workflow

Fig. 13.2 shows the main user workflow, a GPU rental from listing to settlement. The steps are:

1. The provider registers, stakes SMESH, and is attested by the attester. It then publishes a listing with a price per interval.
2. The requester chooses a GPU in the Marketplace, sets the number of intervals and a workload manifest, approves the escrow, and requests the lease.
3. The provider agent checks the manifest against its allowlist and accepts the lease, which bonds part of its stake and starts the clock.
4. The agent runs the workload in a constrained container and sends one heartbeat transaction per interval.
5. Each interval with a heartbeat can be settled, which pays the provider. If an interval has no heartbeat, the lease ends: the remaining escrow is refunded and the bond is slashed.
6. When the lease ends normally, the bond returns to the provider's free stake and any unused escrow returns to the requester.

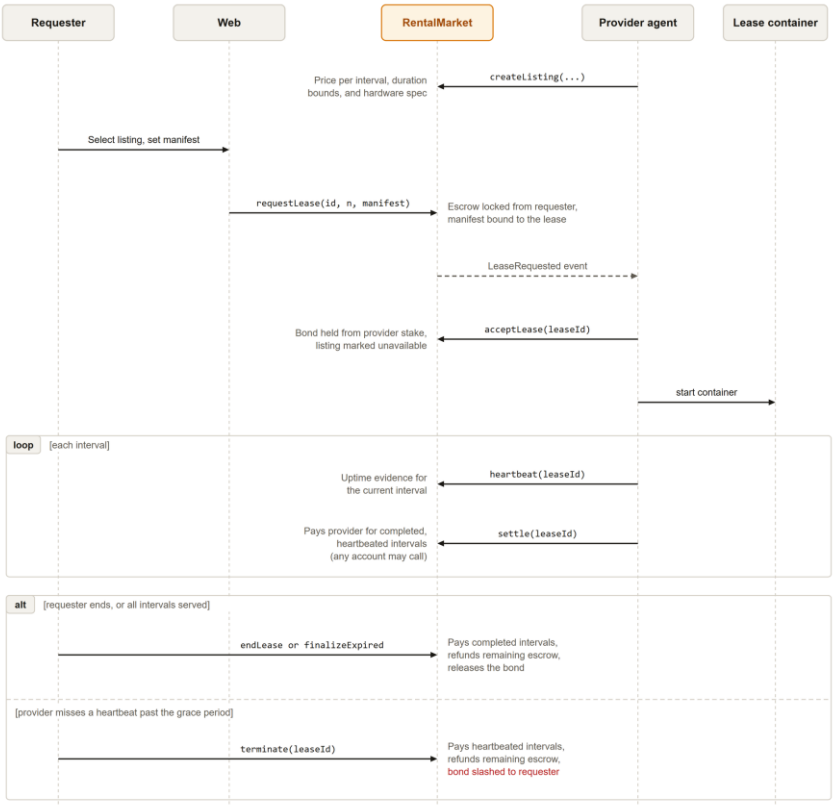


Fig. 13.2: Rental workflow: listing, lease, heartbeats, settlement, and the two ways a lease ends

13.3 Benchmark challenge workflow

Fig. 13.3 shows the benchmark challenge. The attester issues a challenge with a seed derived from the block's randomness, the provider computes the hash chain and measures throughput, and the attester judges the answer. A challenge that is not answered or not judged within the window can be expired by anyone.

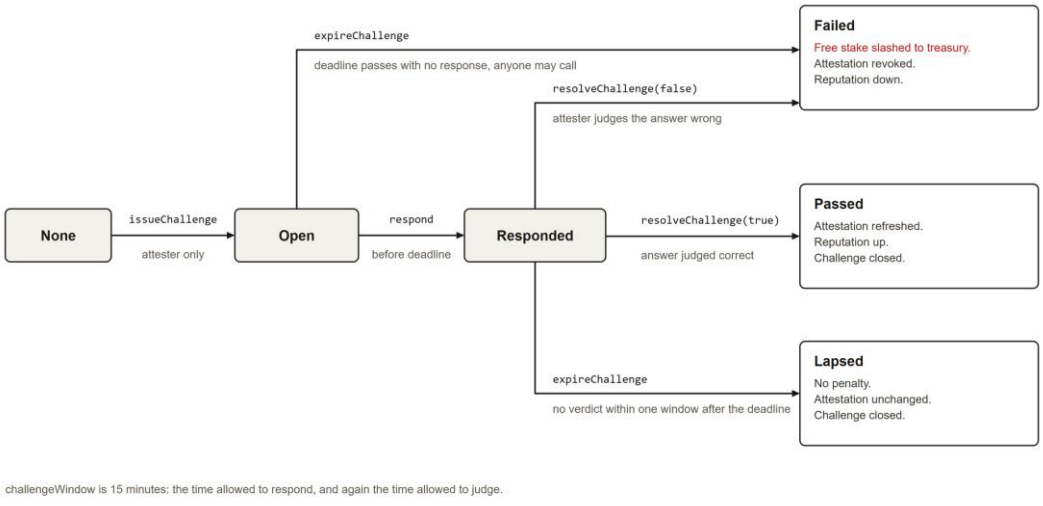


Fig. 13.3: Benchmark challenge states and transitions

14. DATASET AND INPUT DETAILS

The project does not train a model and does not use a fixed dataset. Its inputs are produced by users and by the network, as listed in Table 14.1.

TABLE 14.1: System inputs

Input	Source	Format and use
Hardware descriptor	Provider agent	GPU model, VRAM, region, and models served; registered on chain as capabilities
Workload manifest	Requester	IPFS content identifier of a container manifest; bound to the lease and checked against the provider's allowlist
Inference prompt	Requester	OpenAI-compatible chat request, signed with EIP-191 and carrying a unique request ID
Challenge seed	Contract	keccak256 of block randomness, provider address, and challenge ID; input to the SHA-256 hash chain
Lease parameters	Requester and provider	Price per interval, interval length, number of intervals, and bond
Simulator parameters	Project team	Reward, bond, spot-check rate, challenger pool, outage rates; used to evaluate the economics

15. EXPECTED OUTCOMES

Table 15.1 lists the expected outcomes with the status reached at the time of this report.

TABLE 15.1: Expected outcomes and status

Outcome	Status
Contracts for escrow, stake, rental, inference, verifiable tasks, and benchmark challenges	Done: six contracts, 42 passing tests
Provider agent that runs GPU containers and LLM inference	Done: 58 passing tests; RTX 4070 Laptop GPU serving qwen3:0.6b
Indexer and API that can be rebuilt from chain events	Done: 54 passing tests with PostgreSQL
Web application for requesters and providers	Done: role workspaces; 30 browser checks including wallet transactions
Deployment on a public test network	Done: all six contracts deployed and source-verified on Sepolia
Benchmark challenges on a live provider	Done: challenges answered and passed on Sepolia, about 289 tokens per second measured
Evaluation of cost and economics	Done: gas report for 27 transactions; simulator with 15 passing tests

Measured gas for the main rental and challenge transactions is shown in Table 15.2. At typical test-network gas prices these costs are small compared to the value of an hour of GPU time.

TABLE 15.2: Gas used by core transactions

Transaction	Gas used
Request lease	251,540
Accept lease	154,207
Heartbeat (per interval)	56,487
Settle interval	59,987
Issue benchmark challenge	129,754
Respond to challenge	76,217
Resolve challenge (pass)	54,309
Expire challenge (slash)	80,193
Open inference credit	227,898

The simulation reproduces the expected break-even spot-check rate of 0.5 when reward and bond are equal, and confirms that rental escrow is always conserved: every token deposited is either paid, refunded, or slashed (Fig. 15.1).

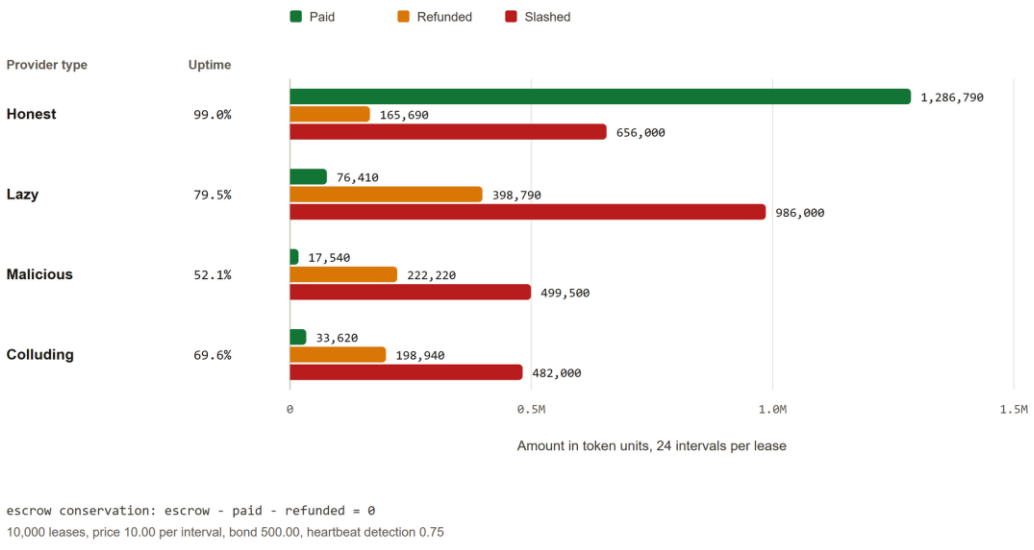


Fig. 15.1: Simulated rental outcomes by provider type: paid, refunded, and slashed

Known limits remain. The attester is a trusted role, LLM output quality is not verified on chain, and a heartbeat proves that the provider's key was live, not the quality of service. These are documented as future work.

16. REFERENCES

[1] Golem Factory GmbH, "Golem overview," official documentation. Accessed: Oct. 2026. [Online]. Available: https://docs.golem.network/docs/golem/overview

[2] iExec, "Proof-of-Contribution," official protocol documentation. Accessed: Oct. 2026. [Online]. Available: https://docs.iex.ec/protocol/proof-of-contribution

[3] Akash Network, "Akash: Decentralized serverless network," official repository and documentation. Accessed: Oct. 2026. [Online]. Available: https://github.com/akash-network/node

[4] Render Foundation, "Render Network whitepaper v4.0," 2023. Accessed: Oct. 2026. [Online]. Available: https://renderfoundation.com/whitepaper

[5] D. Petkanics and E. Tang, "Livepeer whitepaper: Protocol and economic incentives for a decentralized live video streaming network," 2017. Accessed: Oct. 2026. [Online]. Available: https://github.com/livepeer/wiki/blob/master/WHITEPAPER.md

[6] J. Teutsch and C. Reitwiessner, "A scalable verification solution for blockchains," arXiv:1908.04756, 2019. Accessed: Oct. 2026. [Online]. Available: https://arxiv.org/abs/1908.04756

[7] Protocol Labs, "Filecoin: A decentralized storage network," 2017. Accessed: Oct. 2026. [Online]. Available: https://research.protocol.ai/publications/filecoin-a-decentralized-storage-network/

[8] Y. Rao et al., "Bittensor: A peer-to-peer intelligence market," 2021. Accessed: Oct. 2026. [Online]. Available: https://bittensor.com/whitepaper

[9] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten, "Arbitrum: Scalable, private smart contracts," in Proc. 27th USENIX Security Symp., Baltimore, MD, USA, 2018. Accessed: Oct. 2026. [Online]. Available: https://www.usenix.org/conference/usenixsecurity18/presentation/kalodner

[10] Vast.ai, "Vast.ai documentation." Accessed: Oct. 2026. [Online]. Available: https://docs.vast.ai/

[11] Ethereum Foundation, "Proof-of-stake (PoS)," ethereum.org developer documentation. Accessed: Oct. 2026. [Online]. Available: https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/

[12] OpenZeppelin, "OpenZeppelin Contracts," official documentation. Accessed: Oct. 2026. [Online]. Available: https://docs.openzeppelin.com/contracts/

[13] F. Vogelsteller and V. Buterin, "EIP-20: Token standard," Ethereum Improvement Proposals, 2015. Accessed: Oct. 2026. [Online]. Available: https://eips.ethereum.org/EIPS/eip-20

[14] N. Johnson, "EIP-191: Signed data standard," Ethereum Improvement Proposals, 2016. Accessed: Oct. 2026. [Online]. Available: https://eips.ethereum.org/EIPS/eip-191

[15] National Institute of Standards and Technology, "Secure hash standard (SHS)," FIPS PUB 180-4, Aug. 2015. Accessed: Oct. 2026. [Online]. Available: https://csrc.nist.gov/publications/detail/fips/180/4/final

[16] J. Benet, "IPFS: Content addressed, versioned, P2P file system," arXiv:1407.3561, 2014. Accessed: Oct. 2026. [Online]. Available: https://arxiv.org/abs/1407.3561

[17] Nomic Foundation, "Hardhat documentation." Accessed: Oct. 2026. [Online]. Available: https://hardhat.org/docs

[18] Vercel, "Next.js documentation." Accessed: Oct. 2026. [Online]. Available: https://nextjs.org/docs

[19] wevm, "viem: TypeScript interface for Ethereum." Accessed: Oct. 2026. [Online]. Available: https://viem.sh

[20] Ethereum Foundation, "web3.py documentation." Accessed: Oct. 2026. [Online]. Available: https://web3py.readthedocs.io

[21] Docker Inc., "Docker documentation." Accessed: Oct. 2026. [Online]. Available: https://docs.docker.com

[22] Ollama, "Ollama: run large language models locally." Accessed: Oct. 2026. [Online]. Available: https://ollama.com

[23] J. Moxey, E. Marcatilli, and R. Moore, "EIP-1193: Ethereum provider JavaScript API," Ethereum Improvement Proposals, 2018. Accessed: Oct. 2026. [Online]. Available: https://eips.ethereum.org/EIPS/eip-1193

[24] Microsoft, "Playwright documentation." Accessed: Oct. 2026. [Online]. Available: https://playwright.dev/docs/intro